SQLD 제54회 기출 복원문제

문제를 먼저 풀고, 뒤쪽 정답표와 해설로 채점하세요.

문제지 + 정답·해설
총 50문항 · 1과목 10 / 2과목 40

1과목 데이터 모델링의 이해

01 1과목

아래 설명으로 알맞은 데이터 모델링 단계는?

지문

시스템으로 구축하고자 하는 업무에 대해 Key, 속성, 관계 등을 정확하게 표현하고 재사용성이 높은 모델링이다.

① 논리적 데이터 모델링 ② 개념적 데이터 모델링 ③ 물리적 데이터 모델링 ④ 데이터베이스 모델링

02 1과목

엔터티 특징으로 적절하지 않은 것은?

① 엔터티는 고유한 식별자를 가진다.
② 엔터티는 데이터 저장의 기본 단위이다.
③ 엔터티는 반드시 속성을 가진다.
④ 속성이 없는 엔터티가 있을 수 있다.

03 1과목

속성 특징으로 적절하지 않은 것은?

① 속성은 엔터티의 특성을 나타낸다.
② 속성에는 기본 속성과 파생 속성이 있다.
③ 파생 속성은 다른 속성에서 계산된다.
④ 파생 속성은 많을수록 좋다.

04 1과목

식별자의 특징으로 적절하지 않은 것은?

① 식별자는 엔터티의 유일성을 보장한다.
② 식별자는 최소성 조건을 만족해야 한다.
③ 주식별자는 엔터티를 고유하게 식별한다.
④ 주식별자의 값은 자주 변경될 수 있다.

05 1과목

식별자 관계에 대한 설명으로 적절하지 않은 것은?

① 식별자 관계는 부모 엔터티와 종속적인 관계를 맺는다.
② 비식별자 관계에서 부모와의 관계는 항상 필수 조건이다.
③ 식별자 관계는 부모의 식별자를 자식에게 전달한다.
④ 비식별자 관계는 자식 엔터티가 독립적으로 존재할 수 있다.

06 1과목

아래 고객, 주문 관련 ERD에 대한 설명으로 적절한 것은? 주문의 고객번호는 고객 엔터티의 고객번호를 참조한다.

ERD

고객(고객번호, 고객명) 1 : N 주문(주문번호, 고객번호(FK))

① 주문에서의 고객번호는 고객 엔터티에서 상속받은 것이다.
② 고객은 주문 엔터티의 자식 엔터티이다.
③ 주문번호는 고객 엔터티에서 상속받은 식별자이다.
④ 고객과 주문은 서로 아무 관계가 없다.

07 1과목

함수 종속성에 대한 설명으로 가장 적절한 것은?

① 일반 속성이 주식별자 전체에 함수 종속된 상태는 제2정규형의 조건과 관련된다.

② 일반 속성이 주식별자 일부에만 종속되면 제2정규형을 만족한다.

③ 상품번호에 상품명이 종속되는 관계는 상품명이 상품번호를 결정한다는 뜻이다.

④ 함수 종속성은 정규화와 관련이 없다.

08 1과목

아래의 경우 생성하는 식별자는?

지문

업무적으로 만들어지지는 않지만 본질 식별자가 복잡한 구성을 갖고 있기 때문에 인위적으로 만든 식별자

① 외래키 ② 복합 식별자 ③ 인조 식별자 ④ 자연 식별자

09 1과목

관계 표기법에 대한 설명으로 적절하지 않은 것은?

① IE 표기법에서는 관계의 필수 여부를 선과 기호로 표현할 수 있다.

② 바커 표기법에서는 관계의 선택성을 표시할 수 있다.

③ IE 표기법의 실선과 바커 표기법의 O 표시는 같은 의미로 항상 대응된다.

④ 관계 표기법은 엔터티 간 참여도와 관계 수를 표현하는 데 사용된다.

10 1과목

NULL에 대한 설명으로 적절한 것은?

① NULL에는 일반 비교 연산자를 사용해 항상 TRUE 또는 FALSE를 얻을 수 있다.

② 바커 표기법에서는 NULL 허용 여부를 알 수 없다.

③ NULL은 미지의 값으로 일반 비교 연산으로는 비교 결과를 확정하기 어렵다.

④ IE 표기법에서 NULL 허용 여부를 알 수 있다.

2과목 SQL 기본 및 활용

11 2과목

CTAS에 대한 설명으로 적절하지 않은 것은?

① SELECT 결과를 기반으로 새 테이블을 만들 수 있다.

② 제약조건이 모두 복사된다.

③ 기존 테이블의 구조를 기반으로 한다.

④ 일부 제약조건은 별도로 추가할 수 있다.

12 2과목

View에 대한 설명으로 적절하지 않은 것은?

① View는 조회 속도를 높이기 위해 사용된다.

② View는 일반적으로 실제 데이터를 저장하지 않는다.

③ 기본 테이블의 컬럼이 추가되어도 View를 통해 응용 프로그램에 미치는 영향을 줄일 수 있다.

④ View는 다른 테이블의 데이터를 참조한다.

---

13 2과목

Database의 논리적 업무 최소 단위는?

① 트랜잭션 ② 레코드 ③ Query ④ 테이블

---

14 2과목

아래 정규표현식 조건에 해당하지 않는 전화번호 형식으로 가장 적절한 것은?

조건

전화번호는 숫자와 하이픈으로 구성되며, 지역번호가 02로 시작하는 형태를 검사한다.

① 02-123-4567 ② 02-1234-5678 ③ 031-123-4567 ④ 02-9876-5432

---

15 2과목

아래의 계층형 쿼리에 대한 설명으로 적절하지 않은 것은?

```
SELECT emp_no, mgr_no
FROM emp
START WITH mgr_no IS NULL
CONNECT BY PRIOR emp_no = mgr_no;
```

조건

emp_no는 하위 사원 번호, mgr_no는 상위 관리자 번호를 나타낸다.

① 계층 구조를 조회한다. ② 상위 관리자부터 조회한다.

③ 역방향 탐색이다. ④ 순방향 탐색이다.

---

16 2과목

서브쿼리에 대한 설명으로 적절하지 않은 것은?

① 서브쿼리는 메인쿼리 안에서 사용되는 또 다른 SELECT 문이다.

② 서브쿼리의 내부 컬럼은 메인쿼리에서 항상 직접 참조할 수 있다.

③ 상관 서브쿼리는 메인쿼리의 컬럼을 참조할 수 있다.

④ 인라인 뷰는 FROM 절에서 사용되는 서브쿼리이다.

---

17 2과목

아래 CASE 표현식과 같은 결과를 반환하는 함수로 가장 적절한 것은?

```
CASE
WHEN col1 = 'X' THEN NULL
ELSE col1
END
```

① NVL(col1, 'X')

② NULLIF(col1, 'X')

③ COALESCE(col1, 'X')

④ NVL2(col1, NULL, 'X')

18 2과목

다음 SQL 중 Oracle 기준으로 결과가 다른 것은?

TAB

COL1	COL2	COL3
1	2	A
2	3	B
3	2	C
4	3	D

① SELECT COL1, COL2, COL3 FROM TAB WHERE COL1 < 5

② SELECT T.* FROM TAB AS T

③ SELECT * FROM TAB

④ SELECT * FROM TAB WHERE COL2 IN (2, 3)

19 2과목

STADIUM 컬럼이 두 개 테이블에 모두 존재하는 조인 SQL에 대한 설명으로 적절하지 않은 것은?

① 두 테이블에 같은 컬럼명이 있으면 컬럼명 앞에 테이블명 또는 별칭을 붙여 구분해야 한다.

② SELECT 절에서 STADIUM을 사용할 때 모호하면 테이블 별칭을 명시해야 한다.

③ 동일한 이름의 컬럼이 여러 테이블에 존재해도 항상 컬럼명만 단독으로 사용할 수 있다.

④ 조인 SQL에서는 컬럼 소속을 명확히 작성하는 것이 좋다.

20 2과목

아래 SQL에 대한 결과 설명으로 가장 적절한 것은?

```
SELECT COALESCE(A, 50 * B, '50') AS RESULT
FROM T;
```

① B가 NULL이 아니면 항상 B 값을 반환한다.

② A와 B가 NULL일 경우 100을 반환한다.

③ A가 NULL일 경우에는 언제나 50 * B를 반환한다.

④ A가 NULL이 아닐 경우 A 값을 반환한다.

21 2과목

다음 NATURAL JOIN 결과에 대한 설명으로 가장 적절한 것은?

TAB_A

KEYA	COL1	COL2
1	A	가
2	A	가
5	C	다

TAB_B

KEYB	COL1	COL2
1	A	가
3	C	다

```
SELECT *
FROM TAB_A NATURAL JOIN TAB_B;
```

① COL1, COL2가 모두 같은 행이 조인되어 3건이 출력된다.

② KEYA와 KEYB가 같아야만 조인되므로 1건만 출력된다.

③ NATURAL JOIN은 항상 카테시안 곱을 생성한다.

④ 동일 이름 컬럼이 없어 오류가 발생한다.

---

22 2과목

`INNER JOIN` 결과에서 KIM이라는 이름이 중복으로 출력되는 상황에 대한 설명으로 가장 적절한 것은?

① 조인 조건을 만족하는 행 조합이 여러 개이면 같은 값이 중복 출력될 수 있다.

② INNER JOIN은 중복 행을 자동으로 제거한다.

③ INNER JOIN은 항상 한 테이블의 행 수만큼만 출력된다.

④ 중복 이름이 있으면 SQL은 반드시 오류가 발생한다.

---

23 2과목

`LEFT OUTER JOIN` 결과로 SMITH 행의 오른쪽 테이블 컬럼이 NULL로 출력되었다. 이에 대한 설명으로 가장 적절한 것은?

① SMITH 행은 왼쪽 테이블에는 존재하지만 오른쪽 테이블에는 조인 조건을 만족하는 행이 없다.

② SMITH 행은 오른쪽 테이블에만 존재한다.

③ LEFT OUTER JOIN에서는 매칭되지 않은 왼쪽 행이 모두 제거된다.

④ NULL이 출력되면 SQL 문법 오류이다.

---

24 2과목

다음 보기 중 행의 수가 가장 많은 SQL은?

TAB1

CODE	PCOLOR
1	빨강
2	노랑
3	파랑
4	검정

TAB2

CODE	PSIZE
1	소
2	중
3	대
4	특대

① `SELECT CODE FROM TAB1 UNION ALL SELECT CODE FROM TAB2`

② `SELECT * FROM TAB1 JOIN TAB2 ON TAB1.CODE = TAB2.CODE`

③ `SELECT * FROM TAB1, TAB2 WHERE TAB1.PCOLOR IN ('노랑','파랑','검정') AND TAB2.PSIZE IN ('소','중','대')`

④ `SELECT * FROM TAB1 FULL OUTER JOIN TAB2 ON TAB1.CODE = TAB2.CODE`

---

25 2과목

제약 조건에 대한 설명으로 적절하지 않은 것은?

① PK는 NOT NULL이다.

② FK는 부모 테이블의 기본키 또는 고유키를 참조할 수 있다.

③ UK는 항상 NOT NULL이다.

④ CHECK는 특정 조건을 만족하는 값만 허용한다.

---

26 2과목

아래 SQL과 같은 결과를 반환하는 SQL로 가장 적절한 것은?

```
SELECT *
FROM A
WHERE 번호 IN (
SELECT 번호
FROM B
WHERE A.성별 = B.성별
);
```

① `SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE A.성별 = B.성별 AND A.번호 = B.번호)`

② `SELECT * FROM A JOIN B ON A.성별 = B.성별`

③ `SELECT * FROM A WHERE A.성별 IN (SELECT B.성별 FROM B)`

④ `SELECT * FROM A WHERE 번호 = ALL (SELECT 번호 FROM B WHERE A.성별 = B.성별)`

---

27 2과목

아래 SQL 결과로 알맞은 것은?

TAB_A

COL1
1
2
NULL

TAB_B

COL2
1
2
3

```
SELECT COUNT(*)
FROM TAB_B
WHERE COL2 NOT IN (SELECT COL1 FROM TAB_A);
```

① NULL ② 0 ③ 1 ④ 전체 건수

28 2과목

아래 SQL 결과로 알맞은 것은?

TAB

COL
A
B
C
NULL

```
SELECT *
FROM TAB
WHERE COL IN ('A', 'B', NULL);
```

① A, B, C, NULL ② A, B

③ A, B, NULL ④ NULL

29 2과목

아래 SQL 결과로 알맞은 것은?

TAB

나이_그룹	나이
1	10
1	10
2	20
2	20
3	30
3	30
3	NULL

```
SELECT AVG(나이) AS 평균_나이
FROM TAB
GROUP BY 나이_그룹;
```

① 10, 20, 25 ② 10, 20, NULL ③ 10, 20, 30 ④ 10, 20, 20

30 2과목

아래 결과를 출력하기 위한 GROUPING SETS 절로 가장 적절한 것은?

결과

(COL1, COL2)별 상세 집계 행과 COL2별 소계 행만 출력된다. COL1별 소계와 전체 합계 행은 출력되지 않는다.

① `GROUP BY GROUPING SETS ((COL1, COL2), (COL2))`

② `GROUP BY ROLLUP (COL1, COL2)`

③ `GROUP BY CUBE (COL1, COL2)`

④ `GROUP BY GROUPING SETS ((COL1), ())`

31 2과목

아래 Window Function 설명에 맞는 것은?

지문

파티션별 윈도우의 전체 건수에서 현재 행보다 작거나 같은 건수에 대한 누적 백분율을 구하는 함수

① CUME_DIST ② DENSE_RANK ③ RANK ④ PERCENT_RANK

32 2과목

아래 설명을 만족하는 SQL로 알맞은 것은?

지문

부서별로 연봉을 오름차순 정렬하고, 현재 연봉 기준 -100부터 +200 사이의 연봉자 수를 구한다.

① `SELECT emp_id, salary, COUNT(*) OVER (PARTITION BY dep_id ORDER BY salary ROWS BETWEEN 100 PRECEDING AND 200 FOLLOWING) AS salary_count FROM emp`

② `SELECT emp_id, salary, COUNT(*) OVER (PARTITION BY dep_id ORDER BY salary RANGE BETWEEN 100 PRECEDING AND 200 FOLLOWING) AS salary_count FROM emp`

③ `SELECT emp_id, salary, COUNT(*) OVER (PARTITION BY dep_id) AS salary_count FROM emp`

④ `SELECT emp_id, salary, COUNT(*) OVER (ORDER BY salary RANGE BETWEEN 100 PRECEDING AND 200 FOLLOWING) AS salary_count FROM emp`

33 2과목

아래 SQL에서 EMPLOYEE_ID가 7780인 행의 결과로 알맞은 것은?

TAB

EMPLOYEE_ID	SALARY
7780	4000
7781	4000
7782	3000
7783	3000
7784	2000

```
SELECT EMPLOYEE_ID,
ROW_NUMBER() OVER (ORDER BY SALARY, EMPLOYEE_ID) AS ROW_NUM,
RANK() OVER (ORDER BY SALARY) AS RANK_NUM,
DENSE_RANK() OVER (ORDER BY SALARY) AS DENSE_RANK_NUM
FROM EMPLOYEES;
```

① 4, 4, 3 ② 3, 3, 3 ③ 4, 4, 4 ④ 3, 4, 3

34 2과목

강좌번호가 100, 101인 과목을 동시에 듣는 학번을 구하는 SQL로 알맞은 것은?

① `SELECT 학번 FROM 수강 WHERE 강좌번호 = 100 AND 강좌번호 = 101`

② `SELECT 학번 FROM 수강 WHERE 강좌번호 = 100 INTERSECT SELECT 학번 FROM 수강 WHERE 강좌번호 = 101`

③ `SELECT 학번 FROM 수강 WHERE 강좌번호 IN (100, 101)`

④ `SELECT 학번 FROM 수강 WHERE 강좌번호 = 100 OR 101`

35 2과목

아래 결과를 출력하는 SQL의 빈칸에 적절한 것은?

SQL

```
SELECT 이름,
점수,
(SELECT ( 1 )
FROM 점수테이블 T2
WHERE ( 2 )) AS 순위
FROM 점수테이블 T1
ORDER BY 순위;
```

조건

자기보다 점수가 높은 행의 개수에 1을 더해 등수를 구한다.

① (1) COUNT(*) + 1, (2) T1.점수 < T2.점수

② (1) COUNT(*), (2) T1.점수 = T2.점수

③ (1) RANK() + 1, (2) T1.점수 > T2.점수

④ (1) SUM(점수), (2) T1.점수 < T2.점수

---

36 2과목

행이 2건인 테이블에서 결과가 다른 SQL은?

① `SELECT * FROM T WHERE ROWNUM <= 1`

② `SELECT * FROM T WHERE ROWNUM < 2`

③ `SELECT * FROM T WHERE ROWNUM = 1`

④ `SELECT * FROM T WHERE ROWNUM = 2`

---

37 2과목

아래 UPDATE 구문에 대한 설명으로 가장 적절한 것은?

```
UPDATE emp_test A
SET salary = (
SELECT salary * 1.1
FROM emp_test B
WHERE A.employee_id = B.employee_id
AND B.department_id = 60
);
```

조건

> `UPDATE` 문에는 별도의 `WHERE` 절이 없다.

① department_id가 60인 사원들의 월급을 10% 인상하는 쿼리이다.

② `UPDATE ... WHERE department_id = 60`과 동일한 의미이다.

③ 오류가 발생한다.

④ department_id가 60이 아닌 모든 사원들의 salary가 NULL로 수정된다.

---

38 2과목

아래 TABLE에 대한 SQL 수행 후 최종 SUM(COL1) 결과로 가장 적절한 것은?

TABLE

```
CREATE TABLE T (
  COL1 NUMBER PRIMARY KEY,
  COL2 NUMBER CHECK (COL2 > 500)
);
-- 수행 전 T는 비어 있다.
```

SQL

```
INSERT INTO T VALUES (500, 600);
UPDATE T SET COL2 = 400 WHERE COL1 = 500;
INSERT INTO T VALUES (500, 700);
INSERT INTO T VALUES (270, 800);
SELECT SUM(COL1) FROM T;
```

① 770

② 500

③ 0

④ 오류로 인해 모든 작업이 취소된다.

39 2과목

아래 SQL 결과로 알맞은 것은?

```
SELECT COL1,
SUM(매출) AS 총매출
FROM 매출테이블
GROUP BY COL1
ORDER BY 총매출;
```

조건

부서1의 총매출은 18000이고, 부서2의 총매출은 11300이다.

① 부서1 18000, 부서2 11300

② 부서2 11300, 부서1 18000

③ 부서1 11300, 부서2 18000

④ 정렬 순서는 보장되지 않는다.

40 2과목

아래 SQL에 대한 결과로 알맞은 것은?

```
SELECT COL1,
MIN(COL2),
MAX(COL2),
SUM(COL2 + COL3)
FROM T
GROUP BY COL1;
```

조건

COL1이 NULL인 그룹에서는 COL2와 COL3 값이 모두 NULL이다.

① COL1이 NULL인 그룹의 MIN, MAX, SUM 결과는 모두 NULL이다.

② COL1이 NULL인 그룹의 MIN, MAX, SUM 결과는 모두 0이다.

③ COL1이 NULL인 그룹은 GROUP BY 결과에서 제외된다.

④ `SUM(COL2 + COL3)`은 `NULL`을 `0`으로 자동 변환하여 계산한다.

41 2과목

아래 빈칸에 알맞은 명령어는?

지문

입력, 수정, 삭제한 데이터에 대해 전혀 문제가 없다고 판단됐을 경우 ( ) 명령어로 트랜잭션을 완료할 수 있다.

① COMMIT ② ROLLBACK ③ SAVEPOINT ④ REVOKE

42 2과목

아래 SQL에 대한 설명으로 적절한 것은?

```
SELECT 주문번호, 계좌번호
FROM 주문
JOIN 주문내역 ON 주문.주문번호 = 주문내역.주문번호
ORDER BY 계좌번호 DESC;
```

① 계좌번호를 오름차순으로 정렬한다.
② 계좌번호를 내림차순으로 정렬한다.
③ ORDER BY 절이 있으므로 조인이 수행되지 않는다.
④ DESC는 NULL을 제외하는 조건이다.

43 2과목

테이블과 그 테이블에 종속된 제약조건 등 종속 객체까지 함께 삭제하는 명령어로 알맞은 것은?

① TRUNCATE ② DROP CASCADE ③ DELETE ④ ALTER

44 2과목

아래 SQL에 대한 결과로 알맞은 것은? 상품구분이 'C'인 행의 처리결과는?

```
SELECT CASE
WHEN 상품구분 = 'A' THEN '취급'
WHEN 상품구분 = 'B' THEN '보류'
ELSE '취급안함'
END AS 처리결과
FROM 상품;
```

① 취급 ② 보류 ③ 취급안함 ④ NULL

45 2과목

아래 SQL에 대한 설명으로 알맞은 것은?

```
SELECT *
FROM T
WHERE COL1 LIKE 'A%';
```

① COL1이 대문자 A로 시작하는 모든 행을 조회한다.
② COL1이 A로 끝나는 모든 행을 조회한다.
③ COL1에 A가 포함되지 않은 행을 조회한다.
④ COL1의 두 번째 문자가 A인 행을 조회한다.

46 2과목

아래 SQL 결과를 출력하는 SQL로 알맞은 것은?

DATA

COL1	COL2
1	NULL
2	1
3	2

조건

COL1 기준으로 정렬했을 때 이전 행의 COL1 값을 COL2처럼 출력한다.

① `SELECT COL1, LAG(COL1) OVER (ORDER BY COL1) AS COL2 FROM T`

② `SELECT COL1, LEAD(COL1) OVER (ORDER BY COL1) AS COL2 FROM T`

③ `SELECT COL1, FIRST_VALUE(COL1) OVER (ORDER BY COL1) AS COL2 FROM T`

④ `SELECT COL1, ROW_NUMBER() OVER (ORDER BY COL1) AS COL2 FROM T`

---

47 2과목

아래 SQL의 결과로 알맞은 것은?

조건

T는 전체 5건이며, col1이 NULL이 아닌 행은 3건, col2의 NULL을 제외한 서로 다른 값은 2개이다.

```
SELECT COUNT(*) + COUNT(col1) + COUNT(DISTINCT col2) AS RESULT
FROM T;
```

① 5 ② 8 ③ 10 ④ 12

---

48 2과목

단일행 함수에 대한 설명으로 적절하지 않은 것은?

① 각 행에 개별적으로 적용된다.

② 여러 인자를 넣을 수 있다.

③ 중첩 사용이 가능하다.

④ `GROUP BY` 절에는 사용할 수 없다.

---

49 2과목

`INNER JOIN`에 대한 설명 중 적절하지 않은 것은?

① 조인 조건을 만족하는 행만 결과에 포함한다.

② ON 절을 사용하여 조인 조건을 명시할 수 있다.

③ 조인으로 사용되는 컬럼은 컬럼명이 반드시 같아야 한다.

④ 조인 조건이 같음 비교이면 등가 조인으로 볼 수 있다.

---

50 2과목

아래 SQL에 대한 결과로 알맞은 것은?

T1

col1	col2	col3
가	A	2
나	B	3
다	C	NULL

T2

col1	col2
2	A
3	C

```
SELECT *
FROM T1 INNER JOIN T2 ON T1.COL2 = T2.COL2
WHERE 1 = 1
AND T1.COL3 >= 3
AND T2.COL2 IN ('A', 'B');
```

① 결과 없음 ② 가 ③ 나 ④ 다

빠른 정답표

1 ①	2 ④	3 ④	4 ④	5 ②	6 ①	7 ①	8 ③	9 ③	10 ③
11 ②	12 ①	13 ①	14 ③	15 ③	16 ②	17 ②	18 ②	19 ③	20 ④
21 ①	22 ①	23 ①	24 ③	25 ③	26 ①	27 ②	28 ②	29 ③	30 ①
31 ①	32 ②	33 ①	34 ②	35 ①	36 ④	37 ④	38 ①	39 ②	40 ①
41 ①	42 ②	43 ②	44 ③	45 ①	46 ①	47 ③	48 ④	49 ③	50 ①

해설

01 정답 ①

논리적 데이터 모델링은 업무에서 필요한 데이터의 키, 속성, 관계를 구체적으로 표현하고 재사용성이 높은 데이터 구조를 만드는 단계이다.

① 업무 데이터의 키, 속성, 관계를 논리적으로 정리하는 단계이므로 적절하다.

② 개념적 데이터 모델링은 핵심 엔터티와 관계를 높은 추상화 수준에서 파악하는 단계이다.

③ 물리적 데이터 모델링은 DBMS, 인덱스, 저장 구조 등 물리 구현을 고려하는 단계이다.

④ 데이터베이스 모델링은 포괄적 표현이며 단계 명칭으로는 부정확하다.

> 개념 모델링은 업무 개념을 추상화하고, 논리 모델링은 속성·식별자·관계를 구체화한다. 물리 모델링은 실제 DBMS 구현 구조로 변환하는 단계이다.

02 정답 ④

엔터티는 업무상 관리해야 하는 대상이며, 이를 설명하는 속성을 가져야 한다.

① 엔터티는 인스턴스를 식별할 수 있는 식별자를 가져야 한다.

② 엔터티는 업무 데이터를 저장하고 관리하는 기본 단위이다.

③ 엔터티는 하나 이상의 속성을 가져야 한다.

④ 속성이 없는 엔터티는 관리할 데이터가 없으므로 부적절하다.

> 엔터티는 업무에서 독립적으로 식별 가능한 대상이다. 엔터티는 인스턴스의 집합이며, 각 인스턴스는 속성값을 가진다.

03 정답 ④

파생 속성은 계산으로 얻을 수 있으므로 무분별하게 많을수록 좋은 것이 아니라 중복과 정합성 문제를 고려해야 한다.

① 속성은 엔터티 인스턴스를 설명하는 항목이다.

② 속성은 기본 속성, 설계 속성, 파생 속성 등으로 분류할 수 있다.

③ 파생 속성은 다른 속성으로부터 계산되거나 변형되어 생성된다.

④ 파생 속성이 많으면 데이터 중복과 갱신 불일치가 발생할 수 있다.

> 파생 속성은 조회 성능이나 편의성을 위해 둘 수 있지만, 원천 속성과 값이 불일치하지 않도록 관리해야 한다.

04 정답 ④

주식별자는 인스턴스를 안정적으로 식별해야 하므로 자주 변경되는 값은 적절하지 않다.

① 식별자는 인스턴스를 유일하게 구분한다.

② 식별자는 유일성을 만족하는 최소 속성 집합이어야 한다.

③ 주식별자는 엔터티의 대표 식별자이다.

④ 주식별자는 안정성을 가져야 하므로 값 변경이 잦으면 부적절하다.

> 주식별자의 특징은 유일성, 최소성, 불변성, 존재성이다. 자주 바뀌는 속성은 주식별자 후보로 적합하지 않다.

05 정답 ②

비식별자 관계는 부모 식별자가 자식의 일반 속성 또는 외래키로 포함되는 관계이며, 부모와의 참여가 항상 필수인 것은 아니다.

① 식별자 관계에서는 자식이 부모에 존재 종속적인 경우가 많다.

② 비식별자 관계도 선택 관계가 가능하므로 항상 필수라고 할 수 없다.

③ 식별자 관계에서는 부모 식별자가 자식 주식별자에 포함된다.

④ 비식별자 관계에서는 자식이 자체 식별자를 가지고 독립성이 커질 수 있다.

> 식별자 관계는 부모의 식별자가 자식의 주식별자 일부가 되는 관계이다. 비식별자 관계는 부모의 식별자가 자식의 일반 외래키로 존재한다.

06 정답 ①

주문 엔터티의 고객번호는 고객 엔터티의 식별자를 참조하는 외래키로 볼 수 있다.

① 주문의 고객번호는 고객의 고객번호를 참조하므로 적절하다.

② 일반적으로 주문이 고객의 자식 엔터티이다.

③ 주문번호는 주문 자체를 식별하는 속성이다.

④ 고객과 주문은 1:N 관계를 가질 수 있다.

> 부모 엔터티의 식별자는 관계를 통해 자식 엔터티에 외래키로 전달될 수 있다. 식별자 관계인지 비식별자 관계인지에 따라 자식의 주식별자 포함 여부가 달라진다.

07 정답 ①

제2정규형은 제1정규형을 만족하면서 일반 속성이 복합 주식별자 전체에 완전 함수 종속되어야 한다.

① 완전 함수 종속은 제2정규형과 관련된다.

② 일부 식별자에만 종속되는 것은 부분 함수 종속으로 제2정규형 위반이다.

③ 상품번호에 상품명이 종속된다는 것은 상품번호가 상품명을 결정한다는 뜻이다.

④ 함수 종속성은 정규화의 핵심 판단 기준이다.

> 함수 종속은 X 값이 정해지면 Y 값이 하나로 정해지는 관계이다. 정규화는 부분 함수 종속과 이행 함수 종속을 제거하여 이상 현상을 줄인다.

08 정답 ③

업무 식별자가 복잡하거나 부적절할 때 설계자가 인위적으로 만든 식별자는 인조 식별자이다.

① 외래키는 다른 엔터티를 참조하기 위해 전달받은 식별자이다.

② 복합 식별자는 둘 이상의 속성으로 구성된 식별자이다.

③ 인위적으로 만든 식별자이므로 정답이다.

④ 자연 식별자는 업무적으로 자연스럽게 존재하는 식별자이다.

> 인조 식별자는 편리하지만 업무 의미를 직접 담지 않는다. 인조 식별자를 사용하더라도 업무상 유일성 제약은 별도로 고려해야 한다.

09 정답 ③

IE 표기법과 바커 표기법은 관계의 선택성, 필수성, 식별성을 표현하는 방식이 다르므로 특정 기호를 단순히 같은 의미로 항상 대응시키면 안 된다.

① IE 표기법은 관계의 선택성과 카디널리티를 기호로 표현한다.

② 바커 표기법도 관계 선택성과 필수성을 표현할 수 있다.

③ 서로 다른 표기법의 기호를 단순 대응시키는 설명은 부적절하다.

④ 관계 표기법은 엔터티 간 관계 구조를 표현하는 데 사용된다.

> ERD 표기법은 바커 표기법, IE 표기법 등 여러 방식이 있다. 표기법마다 선택 관계, 필수 관계, 식별 관계를 표현하는 기호가 다르므로 해석 기준을 구분해야 한다.

10 정답 ③

NULL은 알 수 없거나 존재하지 않는 값을 의미하며, 일반 비교 연산의 결과가 UNKNOWN이 될 수 있다.

① NULL 비교 결과는 TRUE/FALSE가 아니라 UNKNOWN이 될 수 있다.

② 바커 표기법은 속성 앞 선택성 표기(o)로 NULL 허용 여부를 표현할 수 있으므로 옳지 않은 설명이다.

③ NULL의 의미와 비교 특성을 가장 정확히 설명한다.

④ IE 표기법은 속성 수준의 NULL 허용 여부를 표기하지 않으므로 옳지 않은 설명이다.

> NULL 여부는 = NULL이 아니라 IS NULL 또는 IS NOT NULL로 판단한다. WHERE 절은 TRUE인 행만 반환하므로 UNKNOWN은 결과에서 제외된다.

11 정답 ②

CTAS는 조회 결과를 기반으로 테이블을 생성하지만 기본키, 외래키, CHECK, DEFAULT 등의 제약조건이 모두 복사되는 것은 아니다.

① CTAS는 SELECT 결과로 새 테이블을 생성한다.

② 제약조건이 모두 복사된다는 설명은 틀리다.

③ 조회 대상 컬럼의 구조를 기반으로 테이블이 생성된다.

④ 필요한 제약조건은 생성 후 별도로 추가할 수 있다.

> CTAS는 빠르게 테이블과 데이터를 복제할 때 사용한다. 그러나 인덱스, 제약조건, 트리거 등은 별도로 생성해야 하는 경우가 많다.

12 정답 ①

일반 View는 SQL을 저장한 논리적 객체에 가까우며, 조회 속도를 높이는 것이 주된 목적이라고 보기 어렵다.

① 일반 View는 조회 편의성, 보안, 독립성 제공 목적이 크며 속도 향상이 보장되지 않는다.

② 일반 View는 실제 데이터를 저장하지 않고 기본 테이블을 참조한다.

③ View를 통해 응용 프로그램이 기본 테이블 구조 변경의 영향을 덜 받을 수 있다.

④ View는 기본 테이블 또는 다른 View의 데이터를 참조할 수 있다.

> View는 논리적 테이블처럼 사용할 수 있는 객체이다. 보안, SQL 단순화, 논리적 데이터 독립성 제공에 활용된다.

13 정답 ①

트랜잭션은 데이터베이스에서 더 이상 분할할 수 없는 논리적 업무 처리 단위이다.

① 논리적 업무 최소 단위로 적절하다.

② 레코드는 테이블의 한 행을 의미한다.

③ Query는 트랜잭션을 구성하는 개별 SQL 문일 수 있지만 논리적 업무 단위로 보기 어렵다.

④ 테이블은 데이터를 저장하는 구조이다.

> 트랜잭션은 여러 SQL 문을 하나의 논리적 단위로 묶을 수 있다. 트랜잭션은 ACID 특성을 만족해야 한다.

14 정답 ③

조건은 02로 시작하는 전화번호 형태이므로 031-123-4567은 해당 조건에 맞지 않는다.

① 02로 시작하는 전화번호 형식이다.

② 02로 시작하는 전화번호 형식이다.

③ 031로 시작하므로 02 시작 조건에 맞지 않는다.

④ 02로 시작하는 전화번호 형식이다.

> 정규표현식에서 ^02는 문자열이 02로 시작함을 의미한다. 하이픈과 숫자 개수 조건이 함께 제시되면 각 자리수를 정확히 확인해야 한다.

15 정답 ③

CONNECT BY PRIOR emp_no = mgr_no는 부모 행의 emp_no가 자식 행의 mgr_no와 같은 행을 찾아 내려가는 순방향 계층 탐색이다.

① CONNECT BY를 사용하므로 계층 구조를 조회한다.
② START WITH mgr_no IS NULL이면 최상위 관리자부터 시작한다.
③ 역방향이라는 설명은 틀리다.
④ 부모에서 자식으로 내려가는 순방향이다.

> 계층형 쿼리에서 PRIOR의 위치가 탐색 방향을 결정한다. PRIOR 부모컬럼 = 자식컬럼은 일반적으로 하위 방향 탐색이다.

16 정답 ②

서브쿼리 내부에서만 정의된 컬럼은 일반적으로 메인쿼리에서 직접 참조할 수 없으며, 인라인 뷰처럼 FROM 절에 별칭을 부여한 경우에만 그 결과 컬럼을 참조할 수 있다.

① 서브쿼리의 기본 설명이다.
② 서브쿼리 내부 컬럼을 항상 직접 참조할 수 있다는 설명은 틀리다.
③ 상관 서브쿼리는 메인쿼리의 컬럼을 참조한다.
④ FROM 절의 서브쿼리는 인라인 뷰라고 부른다.

> 서브쿼리는 SELECT, FROM, WHERE 등 여러 절에서 사용할 수 있다. 사용 위치에 따라 스칼라 서브쿼리, 인라인 뷰, 중첩 서브쿼리 등으로 구분된다.

17 정답 ②

NULLIF(col1, 'X')는 col1과 'X'가 같으면 NULL을 반환하고, 다르면 col1을 반환한다.

① NVL은 col1이 NULL이면 대체값을 반환한다.
② CASE 표현식과 같은 의미이다.
③ COALESCE는 첫 번째 NULL이 아닌 값을 반환한다.
④ NVL2는 첫 번째 인자의 NULL 여부에 따라 두 번째 또는 세 번째 인자를 반환한다.

> NULLIF(expr1, expr2)는 두 값이 같으면 NULL, 다르면 expr1을 반환한다. 특정 값과 같을 때 NULL로 바꾸는 경우 유용하다.

18 정답 ②

Oracle에서는 FROM 절의 테이블 별칭에 AS 키워드를 사용할 수 없으므로 SELECT T.* FROM TAB AS T는 오류가 발생한다.

① COL1이 모두 5보다 작으므로 전체 행이 조회된다.
② Oracle 기준 테이블 별칭에 AS를 사용해 문법 오류가 발생한다.
③ 전체 행이 조회된다.
④ COL2가 2 또는 3인 전체 행이 조회된다.

> Oracle에서는 컬럼 별칭에는 AS를 사용할 수 있지만, 테이블 별칭에는 일반적으로 AS를 쓰지 않는다. FROM TAB T처럼 작성한다.

19 정답 ③

조인 대상 테이블에 동일한 컬럼명이 있으면 컬럼명만 단독으로 사용했을 때 모호한 컬럼 오류가 발생할 수 있다.

① 모호한 컬럼은 테이블명 또는 별칭으로 구분해야 한다.
② SELECT 절에서도 모호한 컬럼은 별칭을 붙여야 한다.
③ 항상 단독 사용 가능하다는 설명은 틀리다.
④ 조인 SQL에서는 명시적 컬럼 참조가 가독성과 정확성을 높인다.

> 여러 테이블을 조인할 때 같은 이름의 컬럼이 존재하면 A.STADIUM, B.STADIUM처럼 소속을 명확히 해야 한다.

20 정답 ④

COALESCE는 왼쪽부터 검사하여 첫 번째 NULL이 아닌 값을 반환하므로 A가 NULL이 아니면 A를 반환한다.

① A가 NULL이 아니면 B와 무관하게 A를 반환한다.

② A와 B가 NULL이면 세 번째 인자인 '50'이 반환될 수 있다.

③ B가 NULL이면 50 * B도 NULL이므로 세 번째 인자로 넘어갈 수 있다.

④ COALESCE의 규칙에 맞다.

> COALESCE(expr1, expr2, ...)는 첫 번째 NULL이 아닌 표현식을 반환한다. 인자 간 데이터 타입 호환성도 고려해야 한다.

21 정답 ①

NATURAL JOIN은 두 테이블의 동일 이름 컬럼인 COL1, COL2를 자동으로 조인 조건으로 사용하므로 (A, 가) 2건과 (C, 다) 1건이 매칭되어 총 3건이 출력된다.

① 동일 이름 컬럼 COL1, COL2 기준으로 3건이 조인된다.

② KEYA와 KEYB는 이름이 다르므로 NATURAL JOIN 조건에 포함되지 않는다.

③ 카테시안 곱은 CROSS JOIN의 특징이다.

④ COL1, COL2라는 동일 이름 컬럼이 있다.

> NATURAL JOIN은 동일 이름 컬럼을 모두 조인 조건으로 사용한다. 의도하지 않은 컬럼이 조건에 포함될 수 있으므로 실무에서는 주의가 필요하다.

22 정답 ①

INNER JOIN은 조인 조건을 만족하는 행 조합을 모두 반환하므로 동일한 이름이 여러 행에서 매칭되면 중복 출력될 수 있다.

① 조인 결과의 중복 발생 원인을 적절히 설명한다.

② INNER JOIN은 DISTINCT가 없으면 중복을 제거하지 않는다.

③ 결과 행 수는 매칭 조합 수에 따라 달라진다.

④ 중복 이름 자체는 오류가 아니다.

> 조인 결과는 조건을 만족하는 행의 조합 수로 결정된다. 중복 제거가 필요하면 DISTINCT나 집계가 필요할 수 있다.

23 정답 ①

LEFT OUTER JOIN은 왼쪽 테이블의 행을 보존하며, 오른쪽에 매칭 행이 없으면 오른쪽 컬럼을 NULL로 채운다.

① LEFT OUTER JOIN의 미매칭 행 처리 방식이다.

② 오른쪽 테이블에만 존재하는 행은 LEFT OUTER JOIN에서 기본적으로 보존되지 않는다.

③ LEFT OUTER JOIN은 왼쪽 행을 제거하지 않고 보존한다.

④ NULL 출력은 정상적인 외부 조인 결과이다.

> LEFT OUTER JOIN은 왼쪽 테이블 기준으로 모든 행을 반환한다. 매칭되는 오른쪽 행이 없으면 오른쪽 컬럼 값은 NULL이 된다.

24 정답 ③

3번은 TAB1에서 3행, TAB2에서 3행을 선택해 조인 조건 없이 조합을 만들므로 3×3=9행이 된다.

① TAB1 4행과 TAB2 4행을 UNION ALL 하므로 8행이다.

② CODE가 모두 1:1로 매칭되므로 INNER JOIN 결과는 4행이다.

③ 선택된 3행과 3행의 조합으로 9행이 되어 가장 많다.

④ CODE가 모두 매칭되므로 FULL OUTER JOIN 결과는 4행이다.

> FROM 절에 여러 테이블을 나열하고 조인 조건이 없으면 카테시안 곱이 발생한다. 결과 행 수는 각 입력 행 수의 곱이다.

25 정답 ③

UNIQUE 제약조건은 중복을 제한하지만 DBMS에 따라 NULL을 허용할 수 있으므로 항상 NOT NULL이라고 할 수 없다.

① 기본키는 유일성과 NOT NULL을 만족한다.

② 외래키는 부모 테이블의 기본키 또는 고유키를 참조할 수 있다.

③ UNIQUE가 항상 NOT NULL이라는 설명은 틀리다.

④ CHECK는 조건식을 통해 값의 범위를 제한한다.

> PRIMARY KEY는 UNIQUE와 NOT NULL 성격을 모두 가진다. UNIQUE는 중복 방지 제약이지만 NULL 처리 방식은 DBMS별로 차이가 있다.

26 정답 ①

원 SQL은 같은 성별의 B 행 중 A.번호와 같은 번호가 존재하는지 확인하므로 EXISTS에 성별과 번호 조건을 모두 넣은 1번과 같은 의미이다.

① IN 조건을 EXISTS로 올바르게 변환한 형태이다.

② 성별만 같으면 조인되어 번호 조건이 빠진다.

③ 번호 상관 조건이 빠져 어떤 B 행이든 성별만 같으면 참이 되므로 원문과 결과가 다르다.

④ ALL은 서브쿼리의 모든 번호와 같아야 하므로 의미가 다르다.

> IN은 값이 서브쿼리 결과 집합에 포함되는지 확인한다. EXISTS로 바꿀 때는 동일한 비교 조건을 상관 조건으로 명확히 작성해야 한다.

27 정답 ②

NOT IN의 목록에 NULL이 포함되면 비교 결과가 UNKNOWN이 되어 조건을 만족하는 행이 없어 COUNT(*) 결과는 0이다.

① COUNT(*)는 결과 행이 없어도 NULL이 아니라 0을 반환한다.

② NULL이 포함된 NOT IN 조건 때문에 반환 행이 없다.

③ 3은 1,2와 다르지만 NULL과의 비교 때문에 UNKNOWN이 되어 제외된다.

④ 전체 건수가 반환되지 않는다.

> NOT IN 서브쿼리 결과에 NULL이 있으면 의도와 달리 결과가 없을 수 있다. NULL 가능성이 있으면 NOT EXISTS를 고려하는 것이 안전하다.

28 정답 ②

IN 목록에 NULL이 포함되어도 COL = NULL은 TRUE가 되지 않으므로 A와 B만 조회된다.

① C와 NULL은 조건을 만족하지 않는다.

② A와 B만 IN 목록의 실제 값과 일치한다.

③ NULL은 IN (..., NULL)로 조회되지 않는다.

④ NULL만 조회되지 않는다.

> NULL 값을 조회하려면 COL IS NULL 조건을 별도로 사용해야 한다. IN ('A','B',NULL)은 COL='A' OR COL='B' OR COL=NULL과 유사하게 동작한다.

29 정답 ③

AVG는 NULL을 제외하고 평균을 계산하므로 그룹 1은 10, 그룹 2는 20, 그룹 3은 30이다.

① NULL을 0으로 포함하여 평균을 낸 값과 혼동한 것이다.

② 그룹 3에는 NULL이 아닌 30 값이 있으므로 NULL이 아니다.

③ NULL을 제외한 평균 계산 결과이다.

④ 그룹 3의 평균을 잘못 계산했다.

> AVG, SUM, MIN, MAX 등 대부분의 집계 함수는 NULL을 제외한다. COUNT(*)는 NULL 여부와 관계없이 행 수를 센다.

30 정답 ①

(COL1, COL2) 상세 집계와 (COL2) 집계만 필요하므로 GROUPING SETS((COL1, COL2), (COL2))가 적절하다.

① 필요한 두 그룹 집합만 정확히 지정한다.

② ROLLUP(COL1, COL2)는 (COL1), ()도 생성한다.

③ CUBE는 모든 조합의 소계를 생성한다.

④ 필요한 (COL1, COL2), (COL2)가 없다.

> GROUPING SETS는 원하는 그룹 조합을 직접 지정할 수 있다. ROLLUP과 CUBE보다 필요한 소계만 선택적으로 만들 때 유용하다.

31 정답 ①

CUME_DIST는 현재 행보다 작거나 같은 행의 누적 분포 비율을 반환한다.

① 누적 분포 비율을 구하는 함수이다.

② DENSE_RANK는 동점 후 순위를 건너뛰지 않는 순위 함수이다.

③ RANK는 동점 후 순위를 건너뛰는 순위 함수이다.

④ PERCENT_RANK는 상대 순위를 0~1 범위로 반환한다.

> 분포 함수에는 CUME_DIST, PERCENT_RANK, NTILE, RATIO_TO_REPORT 등이 있다. CUME_DIST는 누적 분포, PERCENT_RANK는 상대 순위를 계산한다.

32 정답 ②

연봉 값의 범위 -100부터 +200 사이를 기준으로 세야 하므로 ROWS가 아니라 RANGE를 사용하고, 부서별 계산을 위해 PARTITION BY dep_id가 필요하다.

① ROWS는 물리적 행 수 기준이므로 연봉 값 범위 조건에 맞지 않는다.

② 부서별 파티션과 연봉 값 범위 프레임을 올바르게 사용했다.

③ 부서 전체 건수만 계산하고 연봉 범위를 반영하지 않는다.

④ 부서별 파티션이 없어 전체 기준으로 계산된다.

> ROWS는 행 개수 기준, RANGE는 ORDER BY 값의 논리적 범위 기준이다. 급여 ± 범위 같은 문제는 RANGE 프레임을 사용한다.

33 정답 ①

SALARY 오름차순에서 2000이 1번째, 3000 두 행이 2~3번째, 4000 두 행이 4~5번째이다. ROW_NUMBER는 SALARY, EMPLOYEE_ID 기준으로 정렬하므로 7780은 4000 그룹의 첫 번째가 되어 ROW_NUMBER 4, RANK 4, DENSE_RANK 3이다.

① ROW_NUMBER 4, RANK 4, DENSE_RANK 3으로 맞다.

② RANK가 3이 될 수 없다.

③ DENSE_RANK는 4000을 세 번째 distinct 값으로 보므로 3이다.

④ ROW_NUMBER가 3이 될 수 없다.

> ROW_NUMBER는 행마다 고유 번호를 부여한다. RANK는 동점 이후 순위를 건너뛰고, DENSE_RANK는 동점 이후 순위를 건너뛰지 않는다. ROW_NUMBER는 동점이 있으면 추가 정렬 기준을 명시해야 결과가 안정적이다.

34 정답 ②

100번 강좌를 듣는 학번 집합과 101번 강좌를 듣는 학번 집합의 교집합을 구해야 하므로 INTERSECT가 적절하다.

① 한 행의 강좌번호가 동시에 100이면서 101일 수 없다.

② 두 강좌를 모두 듣는 학번을 교집합으로 구한다.

③ 둘 중 하나만 듣는 학번도 포함된다.

④ OR 101은 올바른 조건식이 아니다.

> 동시에 만족해야 하는 조건이 서로 다른 행에 나뉘어 존재할 때는 INTERSECT, GROUP BY HAVING COUNT, 셀프 조인 등을 사용할 수 있다.

35 정답 ①

자신보다 점수가 높은 행의 개수를 세고 1을 더하면 등수를 구할 수 있으므로 COUNT(*) + 1과 T1.점수 < T2.점수가 적절하다.

① 자기보다 높은 점수의 개수 + 1로 순위를 구한다.

② 같은 점수의 개수를 세므로 순위가 아니다.

③ 서브쿼리 내부에서 RANK()를 그렇게 사용할 수 없다.

④ 점수 합계를 구하는 식으로 순위 계산과 다르다.

> 셀프 조인 또는 상관 서브쿼리로 순위를 계산할 수 있다. 현재 행보다 큰 값의 개수에 1을 더하면 내림차순 기준 순위가 된다.

36 정답 ④

Oracle에서 ROWNUM은 반환되는 첫 행부터 1이 부여되므로 ROWNUM = 2 조건은 첫 행이 통과하지 못해 결과가 나오지 않는다.

① 첫 번째 행 1건이 반환된다.

② ROWNUM < 2는 첫 번째 행 1건이 반환된다.

③ ROWNUM = 1은 첫 번째 행 1건이 반환된다.

④ ROWNUM = 2는 일반적으로 0건이 된다.

> ROWNUM은 WHERE 조건을 통과하며 반환되는 순서대로 부여된다. N번째 이후 행을 구하려면 인라인 뷰에서 ROWNUM을 먼저 부여한 뒤 바깥 쿼리에서 필터링해야 한다.

37 정답 ④

UPDATE 문에 별도의 WHERE 절이 없으므로 emp_test 의 모든 행이 갱신 대상이 된다. 각 행마다 상관 서브쿼리가 실행되는데, department_id 가 60인 사원은 서브쿼리가 자기 자신 행을 찾아 salary * 1.1 을 반환하므로 10% 인상되고, 60이 아닌 사원은 조건을 만족하는 행이 없어 스칼라 서브쿼리가 NULL 을 반환해 salary 가 NULL 로 갱신된다.

① 서브쿼리 안의 department_id = 60 은 "값을 가져올 행"을 고르는 조건일 뿐, 갱신 대상을 제한하지 않는다. 이 설명은 UPDATE 문에 WHERE 로 대상을 제한했을 때 맞는 내용이며, 나머지 사원의 salary 까지 NULL 로 바꾸는 본 쿼리를 규정하는 설명으로는 옳지 않다.

② 같은 이유로 다르다. WHERE department_id = 60 을 붙인 쿼리는 60이 아닌 사원을 전혀 건드리지 않지만, 본 쿼리는 그들의 salary 를 NULL 로 만든다.

③ 스칼라 서브쿼리가 행을 찾지 못하면 NULL 을 반환할 뿐 문법 오류가 아니다.

④ 정답. 60이 아닌 사원의 salary 가 NULL 이 되는 것은 대상을 제한한 쿼리에서는 일어나지 않는, 본 쿼리만의 결과다.

> SET 절에 상관 서브쿼리를 쓸 때는 갱신 대상 행을 제한하는 WHERE 절을 반드시 함께 써야 한다. 위 쿼리는 WHERE EXISTS (SELECT 1 FROM emp_test B WHERE A.employee_id = B.employee_id AND B.department_id = 60) 을 추가해야 의도대로 60번 부서만 인상된다.

38 정답 ①

첫 번째 INSERT는 정상 수행되어 COL1=500 행이 저장된다. UPDATE는 COL2를 400으로 바꾸려 하므로 CHECK(COL2 > 500)를 위반해 실패한다. 두 번째 INSERT는 COL1=500이 이미 존재하므로 PRIMARY KEY를 위반해 실패한다. 마지막 INSERT는 정상 수행되어 COL1=270 행이 추가된다. 따라서 최종 SUM(COL1)은 500 + 270 = 770이다.

① 정상 반영된 COL1 값 500과 270의 합이다.

② 마지막 정상 INSERT를 누락한 값이다.

③ 정상 INSERT가 모두 취소되는 상황이 아니다.

④ 개별 DML 오류가 이전의 정상 수행 DML을 자동으로 모두 취소한다고 볼 수 없다.

> 제약조건을 위반한 DML은 실패하며 해당 변경은 반영되지 않는다. 이전에 정상 수행된 문장까지 자동으로 모두 취소되는 것은 아니다.

39 정답 ②

ORDER BY 총매출은 기본 오름차순이므로 총매출이 작은 부서2 11300이 먼저 출력되고, 부서1 18000이 다음에 출력된다.

① 총매출 내림차순 결과이다.

② 총매출 오름차순 정렬 결과이다.

③ 부서별 총매출 값이 서로 바뀌었다.

④ ORDER BY가 있으므로 정렬 기준이 있다.

> ORDER BY에서 정렬 방향을 생략하면 기본값은 ASC이다. 별칭으로 지정한 집계 컬럼을 ORDER BY에서 사용할 수 있다.

40 정답 ①

그룹 내 집계 대상 값이 모두 NULL이면 MIN, MAX, SUM 결과는 NULL이 된다.

① 모든 집계 대상이 NULL인 경우의 결과로 적절하다.

② 집계 함수는 NULL을 자동으로 0으로 바꾸지 않는다.

③ GROUP BY에서 NULL은 하나의 그룹으로 묶일 수 있다.

④ NULL이 포함된 산술 연산은 NULL이 된다.

> 집계 함수는 NULL을 제외하지만, 집계할 값이 모두 NULL이면 SUM, MIN, MAX는 NULL을 반환할 수 있다. COUNT는 대상에 따라 0을 반환할 수 있다.

41 정답 ①

COMMIT은 트랜잭션의 변경 내용을 확정하고 영구 반영하는 명령어이다.

① 트랜잭션을 완료하고 변경 내용을 확정한다.

② ROLLBACK은 변경 내용을 취소한다.

③ SAVEPOINT는 트랜잭션 중간 저장점을 만든다.

④ REVOKE는 권한 회수 명령이다.

> TCL에는 COMMIT, ROLLBACK, SAVEPOINT 등이 있다. COMMIT 이후에는 일반적으로 해당 변경을 ROLLBACK으로 되돌릴 수 없다.

42 정답 ②

ORDER BY 계좌번호 DESC는 계좌번호를 내림차순으로 정렬한다.

① 오름차순은 ASC 또는 정렬 방향 생략 시 기본값이다.

② DESC는 내림차순 정렬을 의미한다.

③ ORDER BY는 조인 결과를 정렬하는 절이다.

④ DESC는 NULL 제외 조건이 아니다.

> ORDER BY는 SELECT 결과의 출력 순서를 결정한다. ASC는 오름차순, DESC는 내림차순이다.

43 정답 ②

DROP CASCADE는 객체 삭제 시 종속된 객체까지 함께 삭제하는 옵션으로 사용할 수 있다.

① TRUNCATE는 테이블 데이터만 빠르게 삭제하고 구조는 유지한다.

② 종속 객체까지 함께 제거하는 의미에 가장 적절하다.

③ DELETE는 행 데이터를 삭제하는 DML이다.

④ ALTER는 객체 구조 변경 명령이다.

> DROP은 객체 자체를 삭제한다. CASCADE 옵션은 종속된 제약조건이나 객체까지 함께 제거할 때 사용된다.

44 정답 ③

CASE 조건 중 상품구분='C'에 해당하는 WHEN이 없으므로 ELSE 절의 '취급안함'이 반환된다.

① 'A'일 때 반환되는 값이다.

② 'B'일 때 반환되는 값이다.

③ 조건에 해당하지 않으므로 ELSE 값이 반환된다.

④ ELSE가 있으므로 NULL이 아니라 '취급안함'이다.

> CASE 표현식은 조건을 위에서부터 평가하고 처음 TRUE인 결과를 반환한다. 일치하는 WHEN이 없고 ELSE가 있으면 ELSE 값을 반환한다.

45 정답 ①

LIKE 'A%'는 A로 시작하고 뒤에 임의의 문자열이 오는 값을 찾는다.

① A로 시작하는 값 조건이다.

② A로 끝나는 조건은 '%A'이다.

③ A가 포함되지 않는 조건은 NOT LIKE '%A%'와 가깝다.

④ 두 번째 문자가 A인 조건은 '_A%'이다.

> LIKE에서 %는 0개 이상의 임의 문자열을 의미하고, _는 정확히 한 글자를 의미한다.

46 정답 ①

LAG는 현재 행 기준 이전 행의 값을 가져오므로 COL1이 1일 때 NULL, 2일 때 1, 3일 때 2를 반환한다.

① 이전 행 값을 가져오는 LAG 사용이 적절하다.

② LEAD는 다음 행 값을 가져온다.

③ FIRST_VALUE는 윈도우의 첫 번째 값을 반환한다.

④ ROW_NUMBER는 행 번호를 반환한다.

> LAG는 이전 행, LEAD는 다음 행의 값을 참조하는 윈도우 함수이다. 시계열 비교나 이전 값 대비 변화량 계산에 자주 사용된다.

47 정답 ③

COUNT(*)는 5, COUNT(col1)은 3, COUNT(DISTINCT col2)는 2이므로 합계는 10이다.

① COUNT(*)만 계산한 값이다.

② COUNT(DISTINCT col2)를 누락한 값이다.

③ 5 + 3 + 2 = 10으로 맞다.

④ NULL 또는 중복 값을 잘못 포함한 값이다.

> COUNT(*)는 전체 행 수를 센다. COUNT(컬럼)은 NULL이 아닌 값만 세고, COUNT(DISTINCT 컬럼)은 NULL을 제외하고 중복을 제거한 개수를 센다.

48 정답 ④

단일행 함수는 SELECT, WHERE, ORDER BY뿐 아니라 GROUP BY 절에서도 사용할 수 있다.

① 단일행 함수는 각 행에 대해 개별적으로 작동한다.

② 함수에 따라 여러 인자를 받을 수 있다.

③ 함수의 인자로 다른 함수를 넣는 중첩 사용이 가능하다.

④ GROUP BY 절에 사용할 수 없다는 설명은 틀리다.

> 단일행 함수는 각 행마다 하나의 결과를 반환한다. 예를 들어 GROUP BY SUBSTR(name, 1, 3)처럼 함수 표현식을 그룹 기준으로 사용할 수 있다.

49 정답 ③

INNER JOIN에서 조인 컬럼의 이름이 반드시 같을 필요는 없으며 ON 절에서 서로 다른 이름의 컬럼도 조건으로 연결할 수 있다.

① INNER JOIN은 조건을 만족하는 행만 반환한다.

② ON 절로 조인 조건을 작성할 수 있다.

③ 컬럼명이 반드시 같아야 한다는 설명은 틀리다.

④ 같은 값 비교 조건이면 등가 조인이다.

> USING이나 NATURAL JOIN은 동일 이름 컬럼을 전제로 하지만, ON 절을 사용하는 일반 INNER JOIN은 컬럼명이 달라도 조인할 수 있다.

50 정답 ①

조인 조건 T1.COL2 = T2.COL2를 만족하는 것은 A와 C이다. A 행은 T1.COL3이 2라서 >= 3을 만족하지 못하고, C 행은 COL3이 NULL이며 T2.COL2 IN ('A','B')도 만족하지 못하므로 최종 결과가 없다.

① 모든 조건을 동시에 만족하는 행이 없다.

② 가 행은 조인은 되지만 COL3 >= 3 조건을 만족하지 않는다.

③ 나 행은 T2에 COL2='B'가 없어 조인되지 않는다.

④ 다 행은 COL3이 NULL이고 T2.COL2가 C라 조건을 만족하지 않는다.